

laQ IoT sensors backend

Description: A scalable real-time backend ingest and storage solution for thousands of IoT laQ sensors.

Client (the IoT laQ Sensor):

Image:



Description: The actual laQ sensor. This sensor will need robust data integrity storage for network outages. Currently this buffers data on an SD card regardless of the network connection state. On network connection, this should sync data that is missing in the database from the SD Card.

This has largely been designed hardware wise. The remaining part would be remote firmware management but this will not be directly handled by the time series database.

The only relevant part is the SD card storing data and how that data is retrieved to the main database. This interface/API has not been implemented

The current design pushes 1 min data of the following:

1. ISOTime
2. NOXindex_0-500
3. RTC_TempC
4. RTC_TempC_Cal
5. SCD41_CO2ppm
6. SCD41_RH

7. SCD41_TempC
8. SCD41_TempC_Cal
9. SEN55_PM10p0_ugpm3
10. SEN55_PM1p0_ugpm3
11. SEN55_PM2p5_ugpm3
12. SEN55_PM4p0_ugpm3
13. SEN55_RH
14. SEN55_TempC
15. SEN55_TempC_Cal
16. SHTC3_RH
17. SHTC3_TempC
18. SHTC3_TempC_Cal
19. VOCindex_0-500
20. WiFi_Signal_dBm (used to debug network issues as wifi has been a persistent issue)
21. device_id (<--This is used for uniqueness)
- 22. data_counter (<-- This is a counter that counts up from 0 on every data point used for retrieving and validating data gaps on network connection to the database)**

Some of the metadata on each timestamp will likely be changed going forward.

Backend:

Description: The ingest and data storage for all of the sensors.

The current design uses postgres with the timescaleDB extension as the base database as that is what we've used in the past with reasonable results.

The api connector is currently planned to be implemented in GO to reduce overhead and costs vs python.

There will likely need to be some sort of connection/queue pub/sub broker via mqtt and/or kafka given the potential peak scale of this project.

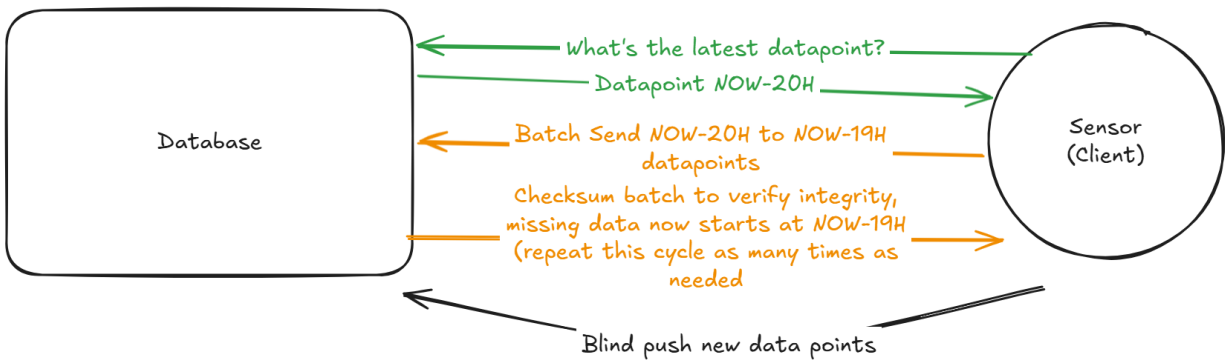
Current design plan is to target at least 10k sensors but should be scalable to millions if needed.

The complicated part:

The main requirement and challenge is how to do the offline data buffering/backfill at scale? No company does this for any public facing entity because it's extremely expensive especially at scale. Govee does this by not storing historical data on the backend and directly connects to the sensor to pull historical data (if the sensor is not online = no historical data).

We want to capture all the data the sensor captures. The **sensor should store data indefinitely on the sdcard** and we want to be able to sync all of it to our backend to keep it in a centralized place. The question becomes **how do we implement this backfill solution to be reliable and robust at scale for thousands upon thousands of sensors? No duplicate data points, no missing data gaps.**

Here is one solution to this based on client pushing missing data on connection:



This has risks of data gaps and **thundering herd reconnection issues** if the backend goes offline and all the sensors reconnect at the same time. That should be handled given this architecture.

Other server-side solutions might need to be investigated that run asynchronously on a cron job to verify data gaps. That “shouldn’t” happen but is possible.

The overall design deliverables for a proof of concept would be:

- TimescaleDB database configuration (schema and hypertable config)
- MQTT broker (this is what the sensors all connect to)
- Kafka queue (This pulls data from mqtt potentially not needed but will depend on load testing)
- API interface written in GO with backfill capability (this pulls data from the queue)
- Security to ensure users/sensors can’t access arbitrary data (authentication layer)
- Observability metrics (db write latency, api latency, queue latency, mqtt latency, connection count, db pooling count, etc)
- Grafana dashboard for viewing observability metrics and live sensor readings
- Some sort of cache layer for serving live data to a front end (Not needed for proof of concept)
- A load generator for this to test the backend to verify it can easily handle ~10k+ sensors from an empty database. (handles steady state and backfill on db disconnect, etc...)
- Ideally this runs on our own hardware no managed services as those become extremely expensive past the free tier.
- Potential horizontal scaling for additional sensor load

Current load is every sensor adds an average of 0.0167 TPS to the database. (1 data point /min).

This is a pretty massive overall project so this would need to be built piece by piece.